Proyecto de Grado: IA al servicio de las comunidades

Maestría en Analítica para la Inteligencia de Negocios

Nombres: María Fernanda Izquierdo Aparicio & Ana María Ochoa Muñoz

Fecha: 18 de Mayo de 2025

Este modelo forma parte del proyecto de grado de la Maestría en Analítica para la Inteligencia de Negocios de la Pontificia Universidad Javeriana, enfocado en la aplicación de Inteligencia Artificial al servicio de las comunidades. Su objetivo es clasificar imágenes de plantas en función de su especie usando una arquitectura pre-entrenada UNet

1. Importe de librerias

In [3]:
```
import json

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.image as mpimg
from pathlib import Path

import os
import zipfile
import shutil
!pip install seaborn
import seaborn as sns
import datetime
!pip install opencv-python
import cv2


import keras
from keras import models
from keras import layers

import random

import tensorflow as tf
```

```
from tensorflow.keras.models import Model
from tensorflow.keras import Layer
from tensorflow.keras.layers import Input, Dense
from tensorflow.keras.preprocessing import image
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.preprocessing.image import img_to_array, load_img
from tensorflow.keras.callbacks import ReduceLROnPlateau
from tensorflow.keras.callbacks import ModelCheckpoint, EarlyStopping, Reduc
from tensorflow.keras.applications import EfficientNetB0
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.layers import Conv2D, MaxPooling2D, UpSampling2D, conc

!pip install scikit-learn

from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, f1_score

from PIL import Image  # Librería para abrir y manipular imágenes

# ignoring warnings
import warnings
warnings.simplefilter("ignore")
```

```
Requirement already satisfied: seaborn in /opt/anaconda3/lib/python3.12/site
-packages (0.13.2)
Requirement already satisfied: numpy!=1.24.0,>=1.20 in /opt/anaconda3/lib/py
thon3.12/site-packages (from seaborn) (1.26.4)
Requirement already satisfied: pandas>=1.2 in /opt/anaconda3/lib/python3.12/
site-packages (from seaborn) (2.2.2)
Requirement already satisfied: matplotlib!=3.6.1,>=3.4 in /opt/anaconda3/li
b/python3.12/site-packages (from seaborn) (3.9.2)
Requirement already satisfied: contourpy>=1.0.1 in /opt/anaconda3/lib/python
3.12/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (1.2.0)
Requirement already satisfied: cycler>=0.10 in /opt/anaconda3/lib/python3.1
2/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (0.11.0)
Requirement already satisfied: fonttools>=4.22.0 in /opt/anaconda3/lib/pytho
n3.12/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (4.51.0)
Requirement already satisfied: kiwisolver>=1.3.1 in /opt/anaconda3/lib/pytho
n3.12/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (1.4.4)
Requirement already satisfied: packaging>=20.0 in /opt/anaconda3/lib/python
3.12/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (24.1)
Requirement already satisfied: pillow>=8 in /opt/anaconda3/lib/python3.12/si
te-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (10.4.0)
Requirement already satisfied: pyparsing>=2.3.1 in /opt/anaconda3/lib/python
3.12/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (3.1.2)
Requirement already satisfied: python-dateutil>=2.7 in /opt/anaconda3/lib/py
thon3.12/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /opt/anaconda3/lib/python3.1
2/site-packages (from pandas>=1.2->seaborn) (2024.1)
Requirement already satisfied: tzdata>=2022.7 in /opt/anaconda3/lib/python3.
12/site-packages (from pandas>=1.2->seaborn) (2023.3)
Requirement already satisfied: six>=1.5 in /opt/anaconda3/lib/python3.12/sit
e-packages (from python-dateutil>=2.7->matplotlib!=3.6.1,>=3.4->seaborn) (1.
16.0)
Requirement already satisfied: opencv-python in /opt/anaconda3/lib/python3.1
2/site-packages (4.11.0.86)
Requirement already satisfied: numpy>=1.21.2 in /opt/anaconda3/lib/python3.1
2/site-packages (from opencv-python) (1.26.4)
Requirement already satisfied: scikit-learn in /opt/anaconda3/lib/python3.1
2/site-packages (1.5.1)
Requirement already satisfied: numpy>=1.19.5 in /opt/anaconda3/lib/python3.1
2/site-packages (from scikit-learn) (1.26.4)
Requirement already satisfied: scipy>=1.6.0 in /opt/anaconda3/lib/python3.1
2/site-packages (from scikit-learn) (1.13.1)
Requirement already satisfied: joblib>=1.2.0 in /opt/anaconda3/lib/python3.1
2/site-packages (from scikit-learn) (1.4.2)
Requirement already satisfied: threadpoolctl>=3.1.0 in /opt/anaconda3/lib/py
thon3.12/site-packages (from scikit-learn) (3.5.0)
```

In [4]:
```python
#Medir_tiempo_ejecucion():
inicio = datetime.datetime.now()
print(f"Inicio: {inicio.strftime('%Y-%m-%d %H:%M:%S.%f')}")
```

```
Inicio: 2025-05-18 21:07:24.256805
```

2. Defínción de Rutas y Parámetros

In [6]:
```
data_dir = 'Datos Finales'  # Reemplaza con la ruta real
metadata_file = 'Metadatos_Finales.csv' # Reemplaza con la ruta real
image_height = 128
image_width = 128
batch_size = 15
num_classes = None  # Se determinará automáticamente del archivo CSV
```

3. Cargar y procesar los metadatos

In [8]:
```
try:
    metadata = pd.read_csv(metadata_file, sep=";")
    metadata = metadata.rename(columns={'Nombre del archivo': 'image_id'})
    metadata = metadata.rename(columns={'Planta_Estado': 'Planta_Estado_Espa

    # Asegúrate de que las columnas 'filename' (o el nombre de tu columna de
    # y 'label' (o el nombre de tu columna de etiquetas) existan.

    if 'image_id' not in metadata.columns or 'Planta_Estado_Español' not in
        raise ValueError("El archivo CSV debe contener las columnas 'image_i
    class_names = metadata['Planta_Estado_Español'].unique().tolist()
    num_classes = len(class_names)
    print(f"Se encontraron {num_classes} clases: {class_names}")
except FileNotFoundError:
    print(f"Error: No se encontró el archivo de metadatos en '{metadata_file
    exit()
except ValueError as e:
    print(f"Error al procesar el archivo CSV: {e}")
    exit()
```

```
Se encontraron 18 clases: ['Yuca_Enferma', 'Yuca_Saludable', 'Fresa_Saludabl
e', 'Papa_Enferma', 'Maíz_Saludable', 'Tomate_Enferma', 'Manzana_Enferma', '
Maíz_Enferma', 'Tomate_Saludable', 'Manzana_Saludable', 'Fresa_Enferma', 'Pa
pa_Saludable', 'Coliflor_Enferma', 'Pepino Cohombro_Saludable', 'Fríjol_Enfe
rma', 'Coliflor_Saludable', 'Fríjol_Saludable', 'Pepino Cohombro_Enferma']
```

In [9]:
```
metadata.Planta_Estado_Español.value_counts()
```

Out[9]:
```
Planta_Estado_Español
Yuca_Enferma                  18820
Tomate_Enferma                16569
Maíz_Enferma                   2690
Yuca_Saludable                 2577
Papa_Enferma                   2000
Manzana_Saludable              1645
Tomate_Saludable               1591
Manzana_Enferma                1529
Maíz_Saludable                 1162
Fresa_Enferma                  1109
Fríjol_Enferma                  868
Fresa_Saludable                 456
Coliflor_Enferma                450
Fríjol_Saludable                428
Pepino Cohombro_Enferma         350
Pepino Cohombro_Saludable       341
Coliflor_Saludable              206
Papa_Saludable                  152
Name: count, dtype: int64
```

In [10]:
```
train, test= train_test_split(metadata, test_size=0.2, random_state=42,shuff
train.shape, test.shape
```

Out[10]:
```
((42354, 12), (10589, 12))
```

In [11]:
```
train.head()
```

Out[11]:

	Dataset	Carpeta	image_i
36463	PlantVillage	Tomato___Tomato_Yellow_Leaf_Curl_Virus	32207a2c-d924-43e7-b466 94832710049d___UF.GRC_
40664	PlantVillage	Tomato___Tomato_Yellow_Leaf_Curl_Virus	87fd49d2-d1b0-4e33-9443 1de82d2c19a1___YLCV_GC
43174	PlantVillage	Tomato___Spider_mites Two-spotted_spider_mite	2fb77f60-ae5b-40c4-b6fb 3c90ac55ccd4___Com.G_S
25436	PlantVillage	Potato___Late_blight	0c2628d4-8d64-48a9-a157 19a9c902e304___RS_LB 4
853	Cassava Leaf Disease Classification	Cassava	1146671365.jp

In [12]:
```
sample = train.sample(5)  # ejemplo común
```

In [13]:
```
print(sample.columns)
```

```
Index(['Dataset', 'Carpeta', 'image_id', 'Planta', 'Enfermedad Inglés',
       'Saludable', 'Planta_Estado_Inglés', 'Planta Español',
       'Planta_Estado_Español', 'Nombre traducido por GPT',
       'Nombre Coloquial en Colombia', 'Nombre Coloquial en TV'],
      dtype='object')
```

4. Crear un generador de datos

In [15]:
```
# Crear el generador de imágenes
datagen = ImageDataGenerator(
    validation_split = 0.2,
    rescale=1./255,
    preprocessing_function = None,
    rotation_range = 45,
    zoom_range = 0.2,
    horizontal_flip = True,
    vertical_flip = True,
    fill_mode = 'nearest',
```

```
    shear_range = 0.1,
    height_shift_range = 0.1,
    width_shift_range = 0.1
)

train_generator = datagen.flow_from_dataframe(
    dataframe=train,
    directory=data_dir,
    x_col='image_id',
    y_col='Planta_Estado_Español',
    target_size=(image_height, image_width),
    batch_size=batch_size,
    class_mode='categorical',  # Importante para clasificación multiclase
    subset='training',
    seed=42,
    classes=class_names  # Asegurarse del orden de las clases
)

validation_generator = datagen.flow_from_dataframe(
    dataframe=train,
    directory=data_dir,
    x_col='image_id',
    y_col='Planta_Estado_Español',
    target_size=(image_height, image_width),
    batch_size=batch_size,
    class_mode='categorical',
    subset='validation',
    seed=42,
    classes=class_names
)
```

```
Found 33880 validated image filenames belonging to 18 classes.
Found 8470 validated image filenames belonging to 18 classes.
```

In [16]:
```
print(train['Planta_Estado_Español'].value_counts())
print(f"Total clases: {train['Planta_Estado_Español'].nunique()}")
```

```
Planta_Estado_Español
Yuca_Enferma                  15056
Tomate_Enferma                13255
Maíz_Enferma                   2152
Yuca_Saludable                 2061
Papa_Enferma                   1600
Manzana_Saludable              1316
Tomate_Saludable               1273
Manzana_Enferma                1223
Maíz_Saludable                  930
Fresa_Enferma                   887
Fríjol_Enferma                  694
Fresa_Saludable                 365
Coliflor_Enferma                360
Fríjol_Saludable                342
Pepino Cohombro_Enferma         280
Pepino Cohombro_Saludable       273
Coliflor_Saludable              165
Papa_Saludable                  122
Name: count, dtype: int64
Total clases: 18
```

In [17]:
```
from tensorflow.keras.preprocessing import image
import os
import numpy as np

img_path = os.path.join(data_dir, metadata["image_id"][20])
img = image.load_img(img_path, target_size=(image_height, image_width))
img_tensor = image.img_to_array(img)
img_tensor = np.expand_dims(img_tensor, axis=0)
img_tensor /= 255.

plt.imshow(img_tensor[0])
plt.axis('off')
plt.show()
```

5. Definición del modelo U-Net para clasificación

In [19]:
```
# DEFINIR EL MODELO U-NET MODIFICADO PARA CLASIFICACIÓN
def unet_classification_model(input_shape=(image_height, image_width, 3), nu
    inputs = Input(shape=input_shape)

    # Encoder
    c1 = Conv2D(32, 3, activation='relu', padding='same')(inputs)
    c1 = Conv2D(32, 3, activation='relu', padding='same')(c1)
    p1 = MaxPooling2D()(c1)

    c2 = Conv2D(64, 3, activation='relu', padding='same')(p1)
    c2 = Conv2D(64, 3, activation='relu', padding='same')(c2)
    p2 = MaxPooling2D()(c2)

    c3 = Conv2D(128, 3, activation='relu', padding='same')(p2)
    c3 = Conv2D(128, 3, activation='relu', padding='same')(c3)
    p3 = MaxPooling2D()(c3)

    # Bottleneck
    c4 = Conv2D(256, 3, activation='relu', padding='same')(p3)
    c4 = Conv2D(256, 3, activation='relu', padding='same')(c4)

    # Decoder
```

```
    u5 = UpSampling2D()(c4)
    u5 = concatenate([u5, c3])
    c5 = Conv2D(128, 3, activation='relu', padding='same')(u5)
    c5 = Conv2D(128, 3, activation='relu', padding='same')(c5)

    u6 = UpSampling2D()(c5)
    u6 = concatenate([u6, c2])
    c6 = Conv2D(64, 3, activation='relu', padding='same')(u6)
    c6 = Conv2D(64, 3, activation='relu', padding='same')(c6)

    u7 = UpSampling2D()(c6)
    u7 = concatenate([u7, c1])
    c7 = Conv2D(32, 3, activation='relu', padding='same')(u7)
    c7 = Conv2D(32, 3, activation='relu', padding='same')(c7)

    # Clasificación
    gap = GlobalAveragePooling2D()(c7)
    outputs = Dense(num_classes, activation='softmax')(gap)

    model = Model(inputs=inputs, outputs=outputs)
    return model


# CREAR Y COMPILAR EL MODELO
model = unet_classification_model(input_shape=(image_height, image_width, 3)
model.compile(optimizer=Adam(), loss='categorical_crossentropy', metrics=['a
model.summary()  # Muestra la arquitectura
```

Model: "functional"

Layer (type)	Output Shape	Param #	Connected to
input_layer (InputLayer)	(None, 128, 128, 3)	0	–
conv2d (Conv2D)	(None, 128, 128, 32)	896	input_layer[0
conv2d_1 (Conv2D)	(None, 128, 128, 32)	9,248	conv2d[0][0]
max_pooling2d (MaxPooling2D)	(None, 64, 64, 32)	0	conv2d_1[0][0
conv2d_2 (Conv2D)	(None, 64, 64, 64)	18,496	max_pooling2d
conv2d_3 (Conv2D)	(None, 64, 64, 64)	36,928	conv2d_2[0][0
max_pooling2d_1	(None, 32, 32,	0	conv2d_3[0][0

(MaxPooling2D)	64)		
conv2d_4 (Conv2D)	(None, 32, 32, 128)	73,856	max_pooling2d
conv2d_5 (Conv2D)	(None, 32, 32, 128)	147,584	conv2d_4[0][0
max_pooling2d_2 (MaxPooling2D)	(None, 16, 16, 128)	0	conv2d_5[0][0
conv2d_6 (Conv2D)	(None, 16, 16, 256)	295,168	max_pooling2d
conv2d_7 (Conv2D)	(None, 16, 16, 256)	590,080	conv2d_6[0][0
up_sampling2d (UpSampling2D)	(None, 32, 32, 256)	0	conv2d_7[0][0
concatenate (Concatenate)	(None, 32, 32, 384)	0	up_sampling2d conv2d_5[0][0
conv2d_8 (Conv2D)	(None, 32, 32, 128)	442,496	concatenate[0
conv2d_9 (Conv2D)	(None, 32, 32, 128)	147,584	conv2d_8[0][0
up_sampling2d_1 (UpSampling2D)	(None, 64, 64, 128)	0	conv2d_9[0][0
concatenate_1 (Concatenate)	(None, 64, 64, 192)	0	up_sampling2d conv2d_3[0][0
conv2d_10 (Conv2D)	(None, 64, 64, 64)	110,656	concatenate_1
conv2d_11 (Conv2D)	(None, 64, 64, 64)	36,928	conv2d_10[0][
up_sampling2d_2 (UpSampling2D)	(None, 128, 128, 64)	0	conv2d_11[0][
concatenate_2 (Concatenate)	(None, 128, 128, 96)	0	up_sampling2d conv2d_1[0][0
conv2d_12 (Conv2D)	(None, 128, 128, 32)	27,680	concatenate_2
conv2d_13 (Conv2D)	(None, 128, 128, 32)	9,248	conv2d_12[0][

global_average_poo… (GlobalAveragePool…	(None, 32)	0	conv2d_13[0][
dense (Dense)	(None, 18)	594	global_averag

Total params: 1,947,442 (7.43 MB)

Trainable params: 1,947,442 (7.43 MB)

Non-trainable params: 0 (0.00 B)

In [20]:
```python
# Earlystopping
from tensorflow.keras.callbacks import EarlyStopping

early_stop = EarlyStopping(
    monitor='val_loss',     # Monitorea la pérdida en validación
    patience=5,             # Número de épocas sin mejora antes de detener
    restore_best_weights=True,  # Restaura los mejores pesos
    verbose=1
)
```

6. Entrenamiento del Modelo

In [22]:
```python
#Entrenamiento
history = model.fit(
    train_generator,
    validation_data=validation_generator,
    epochs=10,
    steps_per_epoch=len(train_generator),
    validation_steps=len(validation_generator),
    callbacks=[early_stop]
)
```

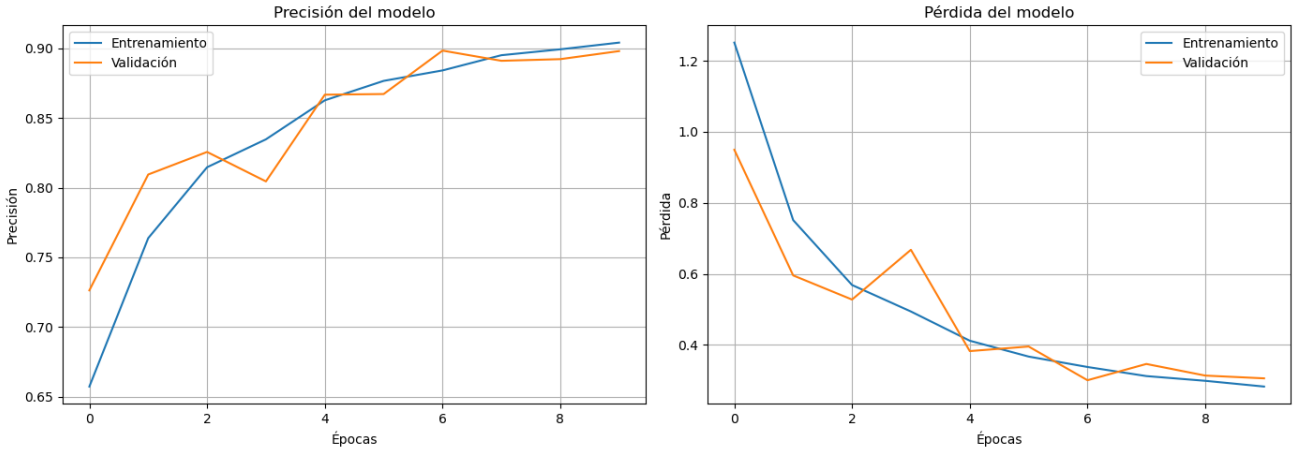
```
Epoch 1/10
2259/2259 ━━━━━━━━━━━━━━━━━━━━ 3042s 1s/step - accuracy: 0.5789 - loss: 1.53
98 - val_accuracy: 0.7263 - val_loss: 0.9493
Epoch 2/10
2259/2259 ━━━━━━━━━━━━━━━━━━━━ 3129s 1s/step - accuracy: 0.7486 - loss: 0.81
56 - val_accuracy: 0.8094 - val_loss: 0.5956
Epoch 3/10
2259/2259 ━━━━━━━━━━━━━━━━━━━━ 3184s 1s/step - accuracy: 0.8033 - loss: 0.59
80 - val_accuracy: 0.8256 - val_loss: 0.5276
Epoch 4/10
2259/2259 ━━━━━━━━━━━━━━━━━━━━ 3122s 1s/step - accuracy: 0.8314 - loss: 0.50
40 - val_accuracy: 0.8045 - val_loss: 0.6677
Epoch 5/10
2259/2259 ━━━━━━━━━━━━━━━━━━━━ 3142s 1s/step - accuracy: 0.8551 - loss: 0.43
61 - val_accuracy: 0.8667 - val_loss: 0.3825
Epoch 6/10
2259/2259 ━━━━━━━━━━━━━━━━━━━━ 3148s 1s/step - accuracy: 0.8734 - loss: 0.37
58 - val_accuracy: 0.8672 - val_loss: 0.3955
Epoch 7/10
2259/2259 ━━━━━━━━━━━━━━━━━━━━ 3178s 1s/step - accuracy: 0.8847 - loss: 0.33
90 - val_accuracy: 0.8983 - val_loss: 0.3005
Epoch 8/10
2259/2259 ━━━━━━━━━━━━━━━━━━━━ 3122s 1s/step - accuracy: 0.8963 - loss: 0.31
11 - val_accuracy: 0.8910 - val_loss: 0.3463
Epoch 9/10
2259/2259 ━━━━━━━━━━━━━━━━━━━━ 3107s 1s/step - accuracy: 0.8975 - loss: 0.30
05 - val_accuracy: 0.8922 - val_loss: 0.3136
Epoch 10/10
2259/2259 ━━━━━━━━━━━━━━━━━━━━ 3113s 1s/step - accuracy: 0.9049 - loss: 0.28
38 - val_accuracy: 0.8980 - val_loss: 0.3058
Restoring model weights from the end of the best epoch: 7.
```

In [23]:
```
fin = datetime.datetime.now()
print(f"Fin: {fin.strftime('%Y-%m-%d %H:%M:%S.%f')}")

duracion = fin - inicio
print(f"Duración: {duracion}")
```

```
Fin: 2025-05-19 05:48:56.272115
Duración: 8:41:32.015310
```

In [24]:
```
import matplotlib.pyplot as plt

# 1. Crear la figura
plt.figure(figsize=(14, 5))

# 2. Gráfico de Accuracy
plt.subplot(1, 2, 1)
plt.plot(history.history['accuracy'], label='Entrenamiento')
plt.plot(history.history['val_accuracy'], label='Validación')
```

```
plt.title('Precisión del modelo')
plt.xlabel('Épocas')
plt.ylabel('Precisión')
plt.legend()
plt.grid(True)

# 3. Gráfico de Loss
plt.subplot(1, 2, 2)
plt.plot(history.history['loss'], label='Entrenamiento')
plt.plot(history.history['val_loss'], label='Validación')
plt.title('Pérdida del modelo')
plt.xlabel('Épocas')
plt.ylabel('Pérdida')
plt.legend()
plt.grid(True)

# 4. Mostrar
plt.tight_layout()
plt.show()
```



In [25]:
```
print(test.columns.tolist())
```

```
['Dataset', 'Carpeta', 'image_id', 'Planta', 'Enfermedad Inglés', 'Saludabl
e', 'Planta_Estado_Inglés', 'Planta Español', 'Planta_Estado_Español', 'Nomb
re traducido por GPT', 'Nombre Coloquial en Colombia', 'Nombre Coloquial en
TV']
```

In [26]:
```
epochs = list(range(1, 11))

loss = history.history['loss']
val_loss = history.history['val_loss']
accuracy = history.history['accuracy']
val_accuracy = history.history['val_accuracy']


data = {
    'Epoch': epochs,
```

```
    'Trainning Loss': loss,
    'Validation Loss': val_loss,
    'Training Accuracy': accuracy,
    'Validation  Accuracy': val_accuracy
}
df = pd.DataFrame(data)

# Imprimir la tabla
print(df)
```

```
   Epoch  Trainning Loss  Validation Loss  Training Accuracy  \
0      1        1.250971         0.949284           0.657290
1      2        0.751224         0.595554           0.763784
2      3        0.568586         0.527622           0.814640
3      4        0.494019         0.667714           0.834681
4      5        0.411758         0.382548           0.862662
5      6        0.367128         0.395457           0.876623
6      7        0.337802         0.300522           0.884120
7      8        0.312214         0.346339           0.895041
8      9        0.298891         0.313617           0.899262
9     10        0.282719         0.305830           0.904073

   Validation  Accuracy
0              0.726328
1              0.809445
2              0.825620
3              0.804486
4              0.866706
5              0.867178
6              0.898347
7              0.891027
8              0.892208
9              0.897993
```

In [27]:
```
print('Perdida final en Train:', history.history['loss'][-1])
print("Pérdida final en Validation:", history.history['val_loss'][-1])

print("Accuracy final en Train:", history.history['accuracy'][-1])
print("Accuracy final en Validation:", history.history['val_accuracy'][-1])
```

```
Perdida final en Train: 0.28271934390068054
Pérdida final en Validation: 0.30582955479621887
Accuracy final en Train: 0.904073178768158
Accuracy final en Validation: 0.8979929089546204
```

In [28]:
```
#validación con datos de test

test_generator =  datagen.flow_from_dataframe(test,
                          directory = "Datos Finales",
                          subset = "training",
                          x_col = "image_id",
```

```
                                y_col = "Planta_Estado_Español",
                                target_size = (image_height,image_width),
                                batch_size = batch_size,
                                class_mode = "categorical")
```

```
Found 8472 validated image filenames belonging to 18 classes.
```

7. Testeo del Modelo

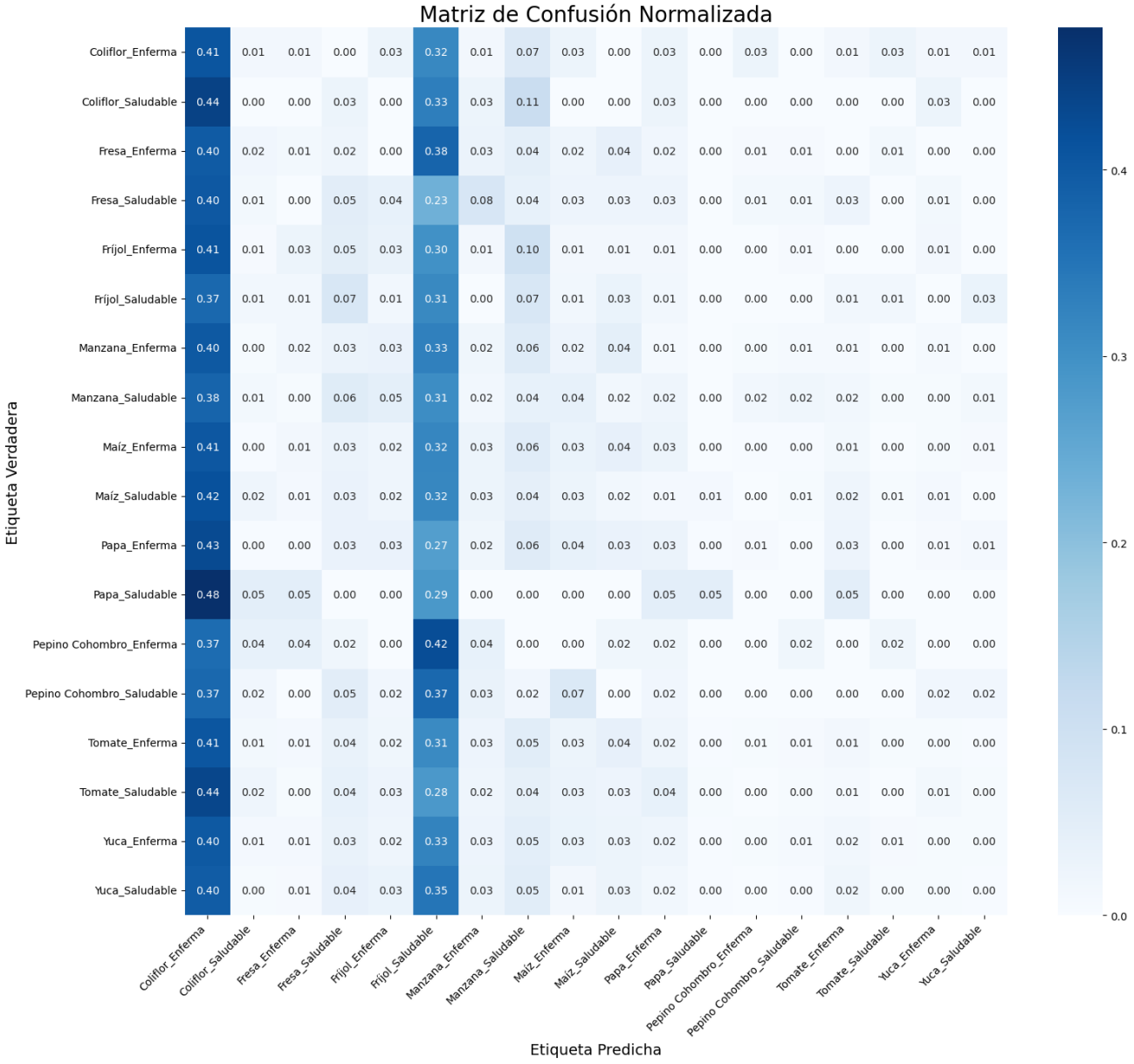
In [30]:
```
# Evaluar sobre el set de test
test_loss, test_accuracy = model.evaluate(test_generator, steps=len(test_ger

print(f'🔎 Pérdida en test: {test_loss:.4f}')
print(f'✅ Precisión en test: {test_accuracy:.2%}')
```

```
565/565 ━━━━━━━━━━━━━━━━━━━━ 221s 391ms/step - accuracy: 0.0330 - loss: 16.3
495
🔎 Pérdida en test: 16.5068
✅ Precisión en test: 3.48%
```

In [31]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix, classification_report
import seaborn as sns

# 1. Predecir clases sobre el test set
# Obtener las predicciones del modelo
y_pred_probs = model.predict(test_generator, steps=len(test_generator), verb

# Convertir probabilidades a etiquetas (argmax)
y_pred = np.argmax(y_pred_probs, axis=1)

# 2. Obtener etiquetas verdaderas
# Las verdaderas etiquetas están en test_generator.classes
y_true = test_generator.classes

# 3. Obtener los nombres de las clases
class_labels = list(test_generator.class_indices.keys())

# 4. Calcular la matriz de confusión
cm = confusion_matrix(y_true, y_pred)

# 5. Normalizar la matriz de confusión (opcional para mejor visualización)
cm_normalized = cm.astype('float') / cm.sum(axis=1)[:, np.newaxis]

# 6. Graficar la matriz de confusión
plt.figure(figsize=(16, 14))
sns.heatmap(cm_normalized, annot=True, fmt='.2f', cmap='Blues', xticklabels=
```

```
plt.title('Matriz de Confusión Normalizada', fontsize=20)
plt.ylabel('Etiqueta Verdadera', fontsize=14)
plt.xlabel('Etiqueta Predicha', fontsize=14)
plt.xticks(rotation=45, ha='right')
plt.yticks(rotation=0)
plt.tight_layout()
plt.show()

# 7. Reporte de clasificación (precisión, recall, f1-score por clase)
print('\nReporte de Clasificación:')
print(classification_report(y_true, y_pred, target_names=class_labels, digit
```

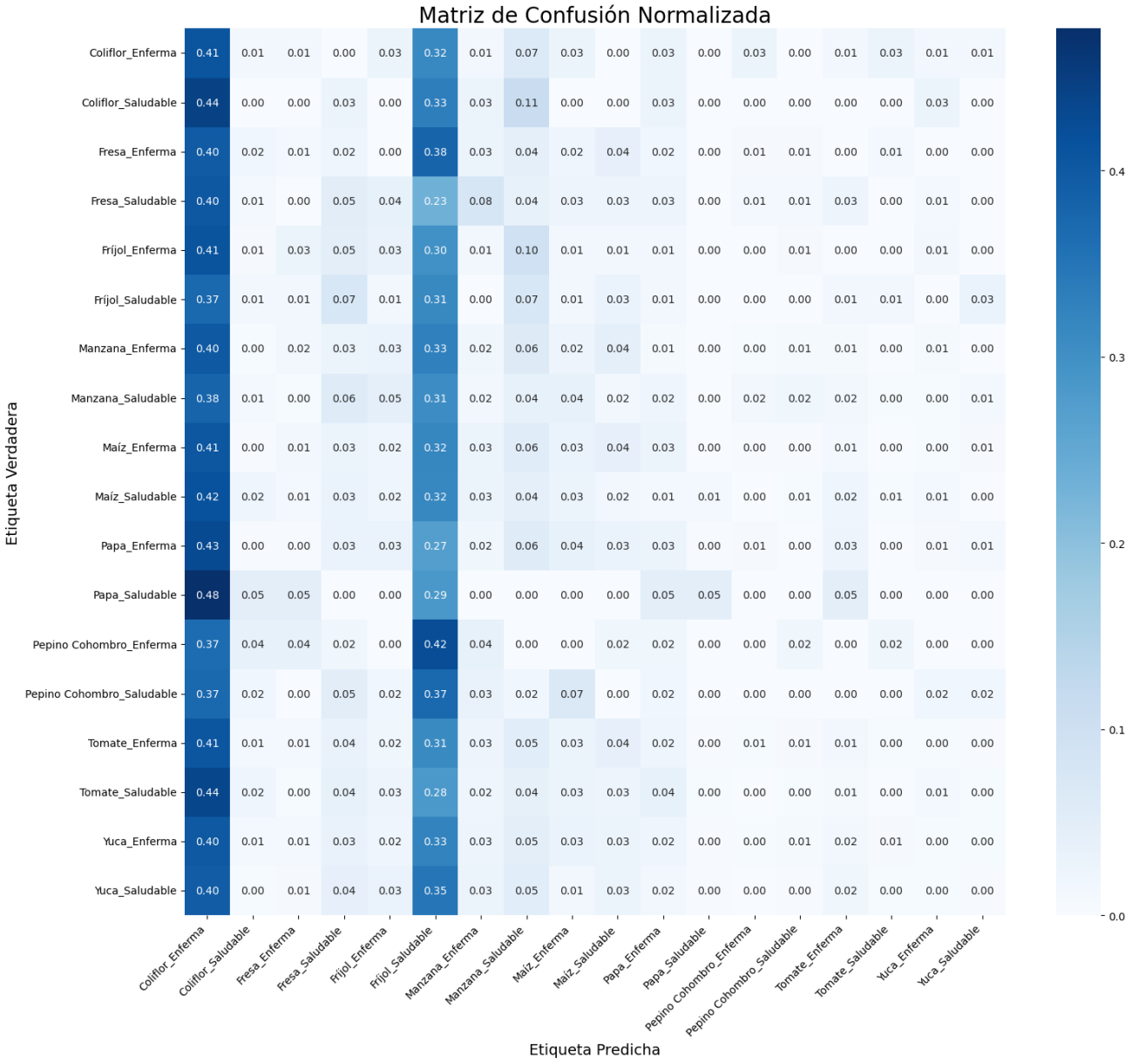
565/565 ━━━━━━━━━━━━━━━━━━━━ **220s** 390ms/step



```
Reporte de Clasificación:
                              precision    recall  f1-score   support

          Coliflor_Enferma     0.0090    0.4079    0.0177        76
        Coliflor_Saludable     0.0000    0.0000    0.0000        36
             Fresa_Enferma     0.0230    0.0112    0.0150       179
           Fresa_Saludable     0.0133    0.0519    0.0212        77
            Fríjol_Enferma     0.0266    0.0347    0.0301       144
          Fríjol_Saludable     0.0078    0.3134    0.0152        67
           Manzana_Enferma     0.0239    0.0204    0.0220       245
         Manzana_Saludable     0.0232    0.0377    0.0287       265
              Maíz_Enferma     0.0544    0.0312    0.0396       417
            Maíz_Saludable     0.0115    0.0164    0.0135       183
              Papa_Enferma     0.0492    0.0288    0.0364       312
            Papa_Saludable     0.0556    0.0476    0.0513        21
  Pepino Cohombro_Enferma     0.0000    0.0000    0.0000        52
Pepino Cohombro_Saludable     0.0000    0.0000    0.0000        59
            Tomate_Enferma     0.3071    0.0146    0.0278      2676
          Tomate_Saludable     0.0000    0.0000    0.0000       236
              Yuca_Enferma     0.2857    0.0040    0.0079      3008
            Yuca_Saludable     0.0270    0.0024    0.0044       419

                  accuracy                         0.0184      8472
                 macro avg     0.0510    0.0568    0.0184      8472
              weighted avg     0.2073    0.0184    0.0183      8472
```

In [32]:

```python
# 1. Guardar la figura de la matriz de confusión
plt.figure(figsize=(16, 14))
sns.heatmap(cm_normalized, annot=True, fmt='.2f', cmap='Blues', xticklabels=
plt.title('Matriz de Confusión Normalizada', fontsize=20)
plt.ylabel('Etiqueta Verdadera', fontsize=14)
plt.xlabel('Etiqueta Predicha', fontsize=14)
plt.xticks(rotation=45, ha='right')
plt.yticks(rotation=0)
plt.tight_layout()

# Guardar el gráfico como imagen
plt.savefig(r'C:\Proyecto de grado\Modelos\matriz_confusion.png', dpi=300)
plt.show()
```



In [33]:
```
# 2. Guardar el reporte de clasificación en CSV

# Convertir el reporte a un diccionario
from sklearn.metrics import classification_report

report = classification_report(y_true, y_pred, target_names=class_labels, ou

# Convertirlo a DataFrame
report_df = pd.DataFrame(report).transpose()

# Guardar como CSV
report_df.to_csv(r'C:\Proyecto de grado\Modelos\reporte_clasificacion.csv',

print("✅ Reporte de clasificación guardado en C:\\Proyecto de grado\\Modelo
```

✅ Reporte de clasificación guardado en C:\Proyecto de grado\Modelos\reporte_clasificacion.csv

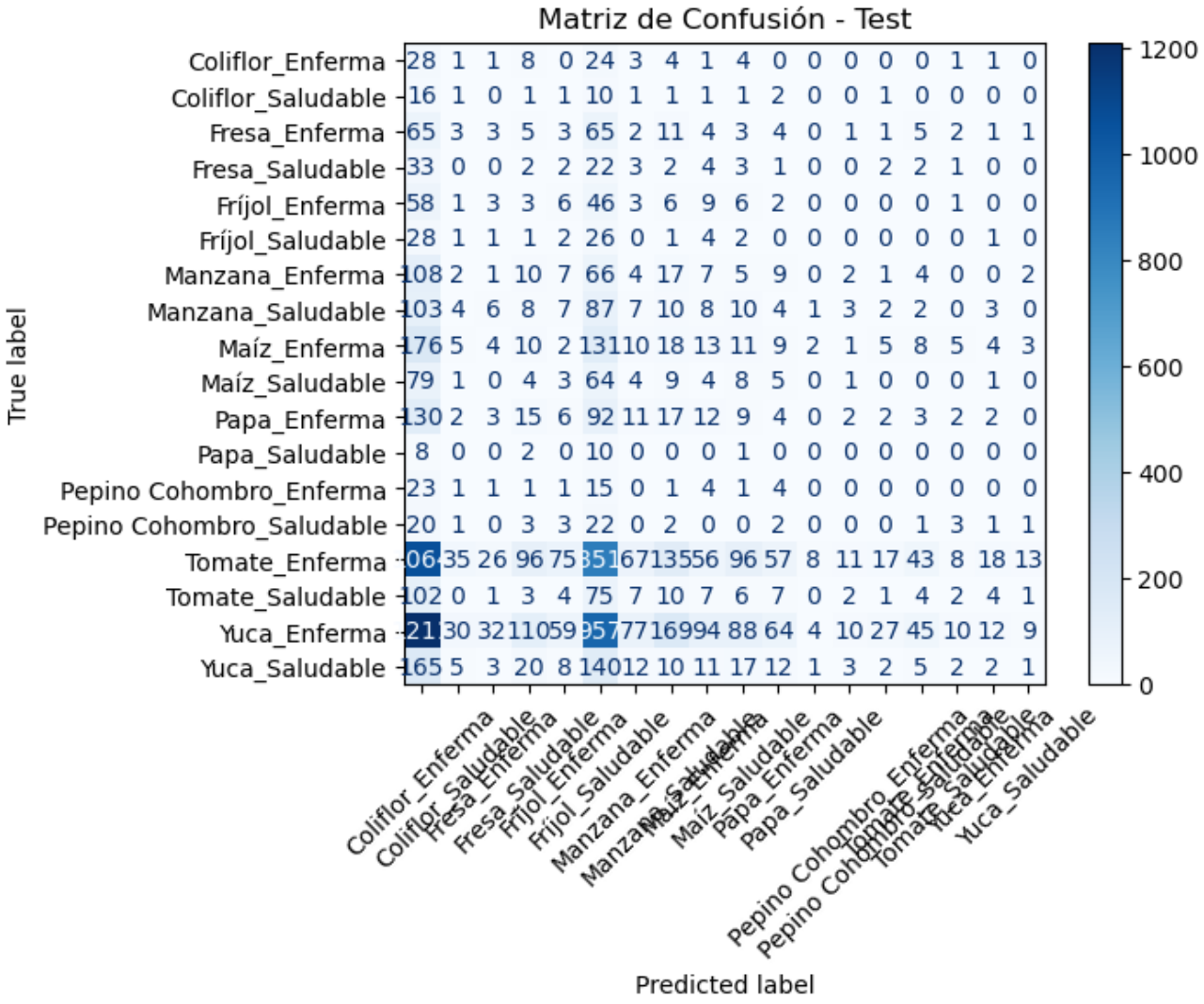
In [34]:
```
accuracy = accuracy_score(y_true, y_pred)
print(f"Test Accuracy: {accuracy:.4f}")
```

```
Test Accuracy: 0.0184
```

In [35]:
```
# ------------------------
# 1. Obtener predicciones completas para test
# ------------------------
y_pred_probs = model.predict(test_generator)
y_pred = np.argmax(y_pred_probs, axis=1)
y_true = test_generator.classes
```

```
565/565 ━━━━━━━━━━━━━━━━━━━━ 220s 390ms/step
```

In [36]:
```
# ------------------------
# 2. Matriz de Confusión
# ------------------------
from sklearn.metrics import confusion_matrix, accuracy_score, ConfusionMatri

cm = confusion_matrix(y_true, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=test_gener
disp.plot(cmap="Blues", xticks_rotation=45)
plt.title("Matriz de Confusión - Test")
plt.show()
```



In [37]:
```python
# Función para normalizar imágenes solo para visualización
def normalize_for_display(img):
    img_min = img.min()
    img_max = img.max()
    if img_max > 1.0 or img_min < 0.0:
        img = (img - img_min) / (img_max - img_min + 1e-7)
    return img

# Obtener nombres de clases
class_indices = test_generator.class_indices
class_names = list(class_indices.keys())

# Reiniciar el generador para empezar desde el principio
test_generator.reset()
x_batch, y_true_batch = next(test_generator)

# Obtener predicciones del modelo
y_pred_probs_batch = model.predict(x_batch)
y_pred_batch = np.argmax(y_pred_probs_batch, axis=1)
```

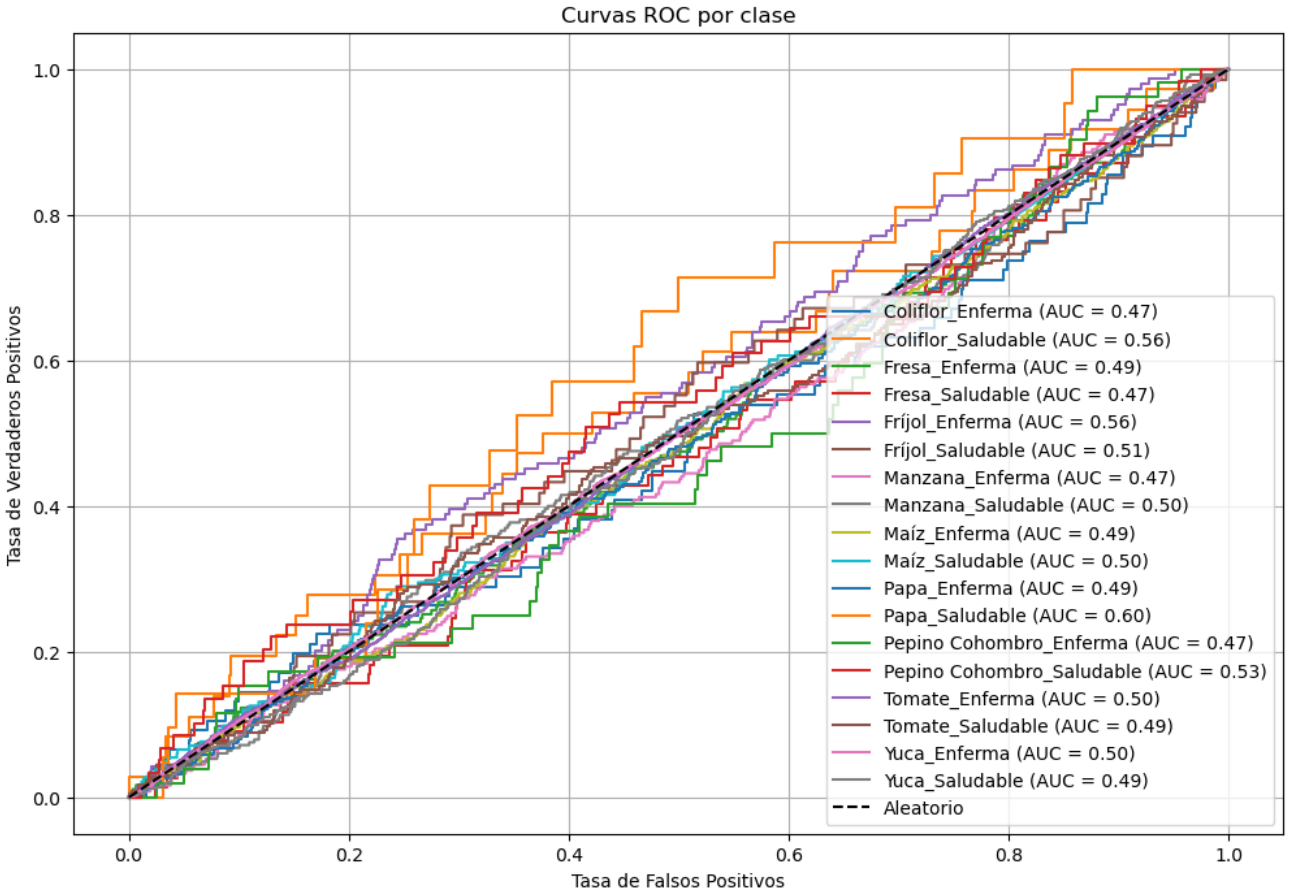
```
y_true_batch = np.argmax(y_true_batch, axis=1)

# Número de imágenes a mostrar (máximo el tamaño del batch)
num_samples = x_batch.shape[0]

# Mostrar imágenes con etiquetas
plt.figure(figsize=(15, 8))
for i in range(num_samples):
    plt.subplot(3, 4, i + 1)
    img_to_show = normalize_for_display(x_batch[i])
    plt.imshow(img_to_show)
    plt.axis('off')
    plt.title(
        f"Real: {class_names[y_true_batch[i]]}\nPred: {class_names[y_pred_ba
        color='green' if y_true_batch[i] == y_pred_batch[i] else 'red'
    )
plt.tight_layout()
plt.suptitle("Muestreo de Predicciones vs. Etiquetas Reales", fontsize=16, y
plt.show()
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 461ms/step
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[37], line 28
     26 plt.figure(figsize=(15, 8))
     27 for i in range(num_samples):
---> 28     plt.subplot(3, 4, i + 1)
     29     img_to_show = normalize_for_display(x_batch[i])
     30     plt.imshow(img_to_show)

File /opt/anaconda3/lib/python3.12/site-packages/matplotlib/pyplot.py:1534,
in subplot(*args, **kwargs)
   1531 fig = gcf()
   1533 # First, search for an existing subplot with a matching spec.
-> 1534 key = SubplotSpec._from_subplot_args(fig, args)
   1536 for ax in fig.axes:
   1537     # If we found an Axes at the position, we can reuse it if the us
er passed no
   1538     # kwargs or if the Axes class and kwargs are identical.
   1539     if (ax.get_subplotspec() == key
   1540         and (kwargs == {}
   1541              or (ax._projection_init
   1542                  == fig._process_projection_requirements(**kwargs)))
):

File /opt/anaconda3/lib/python3.12/site-packages/matplotlib/gridspec.py:589,
in SubplotSpec._from_subplot_args(figure, args)
    587 else:
    588     if not isinstance(num, Integral) or num < 1 or num > rows*cols:
--> 589         raise ValueError(
    590             f"num must be an integer with 1 <= num <= {rows*cols}, "
    591             f"not {num!r}"
    592         )
    593     i = j = num
    594 return gs[i-1:j]

ValueError: num must be an integer with 1 <= num <= 12, not 13
```

In [38]:
```python
from sklearn.metrics import roc_curve, auc
from tensorflow.keras.utils import to_categorical

# 3. Convertir etiquetas reales a one-hot
n_classes = y_pred_probs.shape[1]
y_true_onehot = to_categorical(y_true, num_classes=n_classes)

# 4. Calcular ROC y AUC por clase
fpr = dict()
tpr = dict()
roc_auc = dict()

for i in range(n_classes):
    fpr[i], tpr[i], _ = roc_curve(y_true_onehot[:, i], y_pred_probs[:, i])
    roc_auc[i] = auc(fpr[i], tpr[i])

# 5. Obtener nombres de clase
class_names = list(test_generator.class_indices.keys())

# 6. Graficar curvas ROC
plt.figure(figsize=(12, 8))
for i in range(n_classes):
    plt.plot(fpr[i], tpr[i], label=f'{class_names[i]} (AUC = {roc_auc[i]:.2f

plt.plot([0, 1], [0, 1], 'k--', label='Aleatorio')
plt.xlabel('Tasa de Falsos Positivos')
plt.ylabel('Tasa de Verdaderos Positivos')
```

```
plt.title('Curvas ROC por clase')
plt.legend(loc='lower right')
plt.grid(True)
plt.show()
```



In [40]:
```
from sklearn.metrics import roc_auc_score

# AUC macro (promedio de clases, todas igual de importantes)
auc_macro = roc_auc_score(y_true_onehot, y_pred_probs, average='macro')

# AUC micro (promedia sobre todas las instancias)
auc_micro = roc_auc_score(y_true_onehot, y_pred_probs, average='micro')

print(f"AUC Macro promedio: {auc_macro:.4f}")
print(f"AUC Micro promedio: {auc_micro:.4f}")
```

```
AUC Macro promedio: 0.5042
AUC Micro promedio: 0.5516
```

In [42]:
```
# Asegúrate de tener esto:
# y_pred = np.argmax(y_pred_probs, axis=1)
# y_true = test_generator.classes

# Accuracy
accuracy = accuracy_score(y_true, y_pred)
```

```
# F1 Scores
f1_macro = f1_score(y_true, y_pred, average='macro')
f1_micro = f1_score(y_true, y_pred, average='micro')
f1_weighted = f1_score(y_true, y_pred, average='weighted')

print(f"Accuracy: {accuracy:.4f}")
print(f"F1 Score (macro): {f1_macro:.4f}")
print(f"F1 Score (micro): {f1_micro:.4f}")
print(f"F1 Score (weighted): {f1_weighted:.4f}")

# Detalle por clase
class_names = list(test_generator.class_indices.keys())
print("\nReporte de clasificación por clase:")
print(classification_report(y_true, y_pred, target_names=class_names))
```

```
Accuracy: 0.0192
F1 Score (macro): 0.0175
F1 Score (micro): 0.0192
F1 Score (weighted): 0.0194

Reporte de clasificación por clase:
                           precision    recall  f1-score   support

         Coliflor_Enferma       0.01      0.37      0.02        76
       Coliflor_Saludable       0.01      0.03      0.02        36
            Fresa_Enferma       0.04      0.02      0.02       179
          Fresa_Saludable       0.01      0.03      0.01        77
           Fríjol_Enferma       0.03      0.04      0.04       144
         Fríjol_Saludable       0.01      0.39      0.02        67
          Manzana_Enferma       0.02      0.02      0.02       245
        Manzana_Saludable       0.02      0.04      0.03       265
             Maíz_Enferma       0.05      0.03      0.04       417
           Maíz_Saludable       0.03      0.04      0.04       183
            Papa_Enferma       0.02      0.01      0.02       312
          Papa_Saludable       0.00      0.00      0.00        21
  Pepino Cohombro_Enferma       0.00      0.00      0.00        52
Pepino Cohombro_Saludable       0.00      0.00      0.00        59
           Tomate_Enferma       0.35      0.02      0.03      2676
         Tomate_Saludable       0.05      0.01      0.01       236
             Yuca_Enferma       0.24      0.00      0.01      3008
           Yuca_Saludable       0.03      0.00      0.00       419

                 accuracy                           0.02      8472
                macro avg       0.05      0.06      0.02      8472
             weighted avg       0.21      0.02      0.02      8472
```

8. Guardar el modelo entrenado

In [45]:
```
model.save('modelo_15_Unet_PlantaEstado.h5')
```

```
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file format is considered legacy. We recommend using instead the native Keras format, e.g. `model.save('my_model.keras')` or `keras.saving.save_model(model, 'my_model.keras')`.
```

In [47]:
```
fin = datetime.datetime.now()
print(f"Fin: {fin.strftime('%Y-%m-%d %H:%M:%S.%f')}")

duracion = fin - inicio
print(f"Duración: {duracion}")
```

```
Fin: 2025-05-19 06:11:11.300887
Duración: 9:03:47.044082
```

In [49]:
```
class_indices = train_generator.class_indices  # o test_generator.class_indi
index_to_class = {v: k for k, v in class_indices.items()}
print(index_to_class)
with open("index_to_class_modelo15_NoBalanceado.json", "w") as f:
    json.dump(index_to_class, f, indent=4, ensure_ascii=False)
```

```
{0: 'Yuca_Enferma', 1: 'Yuca_Saludable', 2: 'Fresa_Saludable', 3: 'Papa_Enferma', 4: 'Maíz_Saludable', 5: 'Tomate_Enferma', 6: 'Manzana_Enferma', 7: 'Maíz_Enferma', 8: 'Tomate_Saludable', 9: 'Manzana_Saludable', 10: 'Fresa_Enferma', 11: 'Papa_Saludable', 12: 'Coliflor_Enferma', 13: 'Pepino Cohombro_Saludable', 14: 'Fríjol_Enferma', 15: 'Coliflor_Saludable', 16: 'Fríjol_Saludable', 17: 'Pepino Cohombro_Enferma'}
```

In [ ]:
```

```